ÉCOLE TECHNIQUE – ÉCOLE DES MÉTIERS DE LAUSANNE

PASSION LECTURE

Rapport de projet – P_WEB294

Frontend d'une application de partage de livres

Équipe de projet

Arnaud Thill • Sarangan Shanthakumar • Kentin Fankhauser

Informations générales

Chef de projet : Grégorie Charmier

Période de réalisation : du 28 août au 30 octobre 2026

Lieu : ETML – EPV, Valmont 28b, 1004 Lausanne

Table des matières

1. Introduction 3
2. Analyse 4
2.1 Maquettes de l'application 4
2.2 Planification et suivi des tâches 6
2.3 Organisation du code 7
3. Réalisation 8
3.1 Technologies et navigation 8
3.2 Page d'accueil 8
3.3 Catalogue par catégorie 8
3.4 Vue détaillée et appréciations 8
3.5 Ajouter un livre 8
3.6 Modifier un livre 8
3.7 Supprimer un livre 8
3.8 Gestion des comptes, des droits et pied de page 9
3.9 Organisation du groupe avec Git/GitHub 9
4. Conclusion 10
4.1 Bilan général 10
4.2 Conclusions personnelles 10
4.3 Critique constructive de la planification 10
5. Webographie, bibliographie et glossaire 11
5.1 Sources du projet 11
5.2 Webographie externe 11
5.3 Glossaire 11

1. Introduction

Nous avons travaillé sur Passion Lecture, une application web destinée à présenter des livres et à permettre aux passionnés de lecture de partager leurs découvertes. Nous avons prévu un accueil et un catalogue par catégorie accessibles aux visiteurs. Les membres peuvent proposer leurs propres ouvrages, les gérer, puis donner une appréciation et rédiger un commentaire. Pour les administrateurs, nous avons prévu l'ensemble des actions demandées dans le cahier des charges.

Dans le cadre du module 294 de l'ETML, nous nous sommes concentrés sur la réalisation du frontend. Pour gérer les données, nous avons utilisé une API REST simulée avec JSON Server. Le développement d'un backend complet est prévu ultérieurement dans le module 295. Notre objectif était de proposer une interface compréhensible, structurée et exploitable.

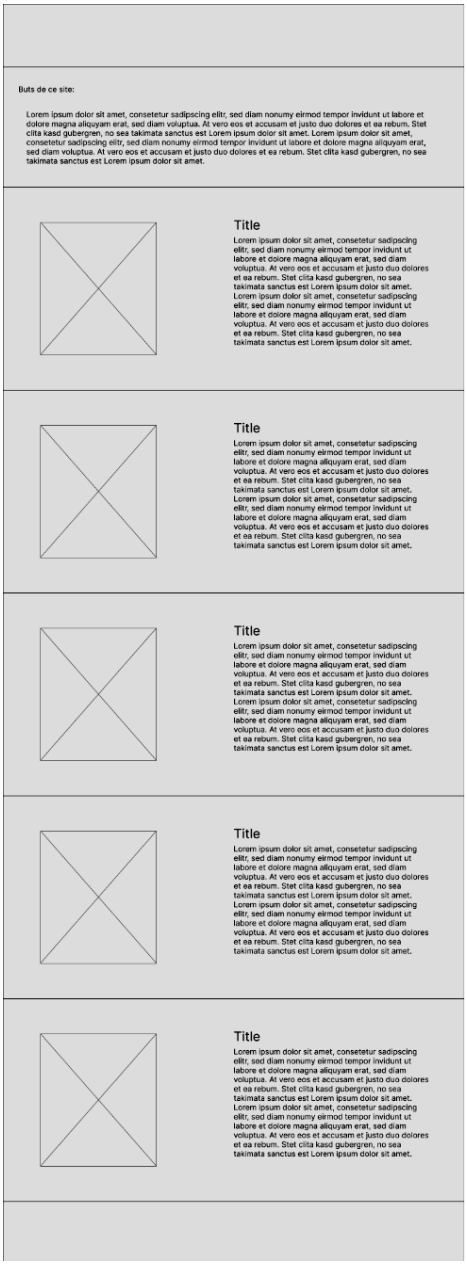
Nous avons réalisé ce projet à trois : Arnaud Thill, Sarangan Shanthakumar et Kentin Fankhauser. Dans ce rapport, nous présentons nos maquettes, notre planification, l'organisation de notre code, les fonctionnalités développées et le bilan de notre travail en équipe, en suivant la structure demandée dans le cahier des charges.

2. Analyse

2.1 Maquettes de l'application

Nous avons conçu quatre maquettes pour représenter les principales pages de notre application et les parcours de navigation. Elles nous ont servi de référence pour organiser les contenus avant de les développer en Vue.js.

Accueil



Catalogue par genre

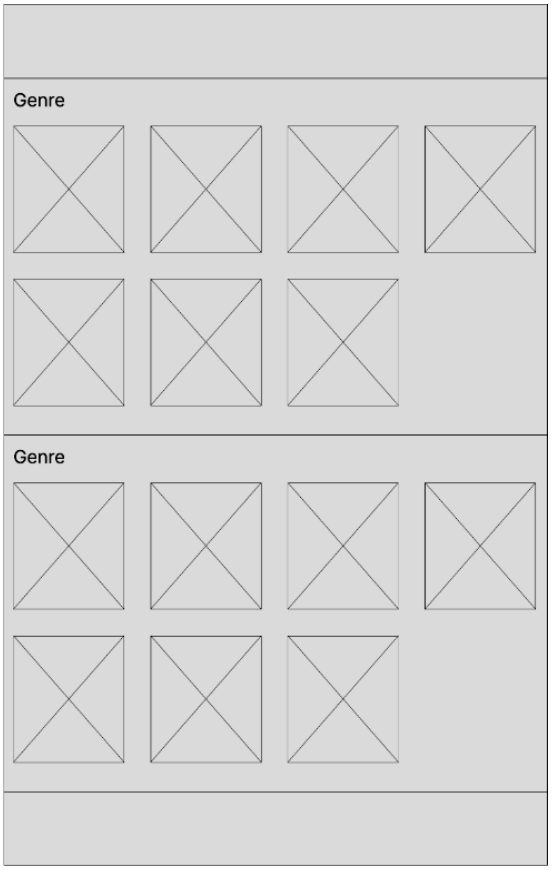
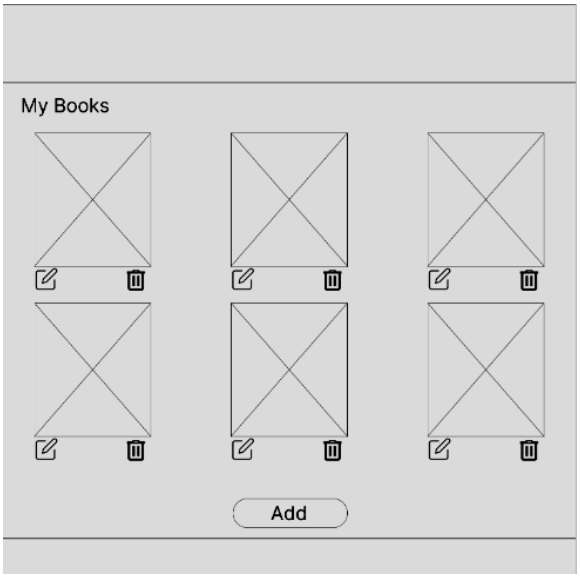


Figure 1 – Maquettes de l'accueil et du catalogue.

Pour l'accueil, nous avons prévu une présentation de l'utilité du site ainsi que les cinq derniers ouvrages. Dans le catalogue, nous avons choisi de regrouper les livres par genre pour faciliter l'accès à leur fiche.

2.1 Maquettes de l'application (suite)

Mes livres



Détail d'un livre

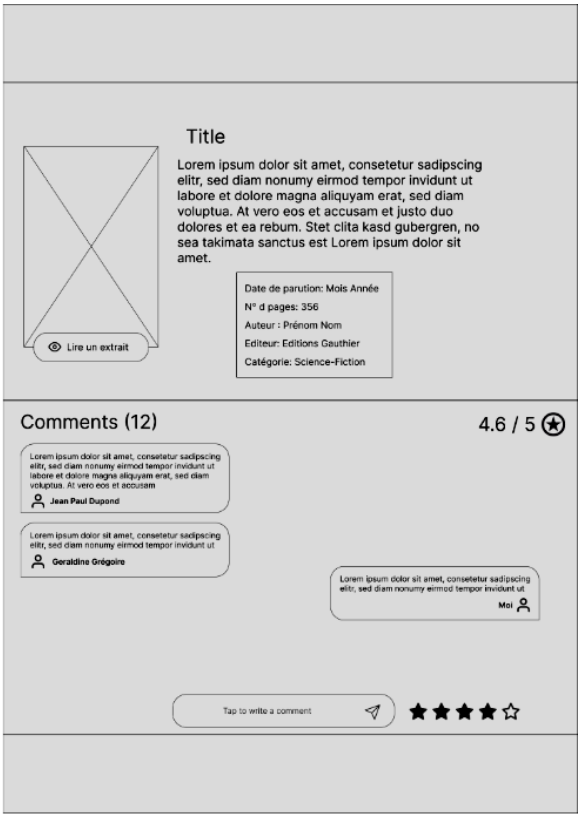


Figure 2 – Maquettes de la gestion des livres et de la consultation détaillée.

Dans la page « Mes livres », nous avons prévu les actions d'ajout, de modification et de suppression. Pour la fiche détaillée, nous avons prévu les informations bibliographiques, un extrait PDF, la note moyenne, les commentaires et un formulaire d'avis.

2.2 Planification et suivi des tâches

Nous avons organisé notre travail dans GitHub Projects avec les colonnes « Todo », « In Progress » et « Done ». Sur nos captures du 30 octobre 2026, nous comptons 19 tâches classées « Done ». Nous n'avons pas retrouvé les tâches #10 (vue détail utilisateur) et #14 (gestion des accès), auparavant en « Todo », parmi les tâches terminées visibles sur ces captures.

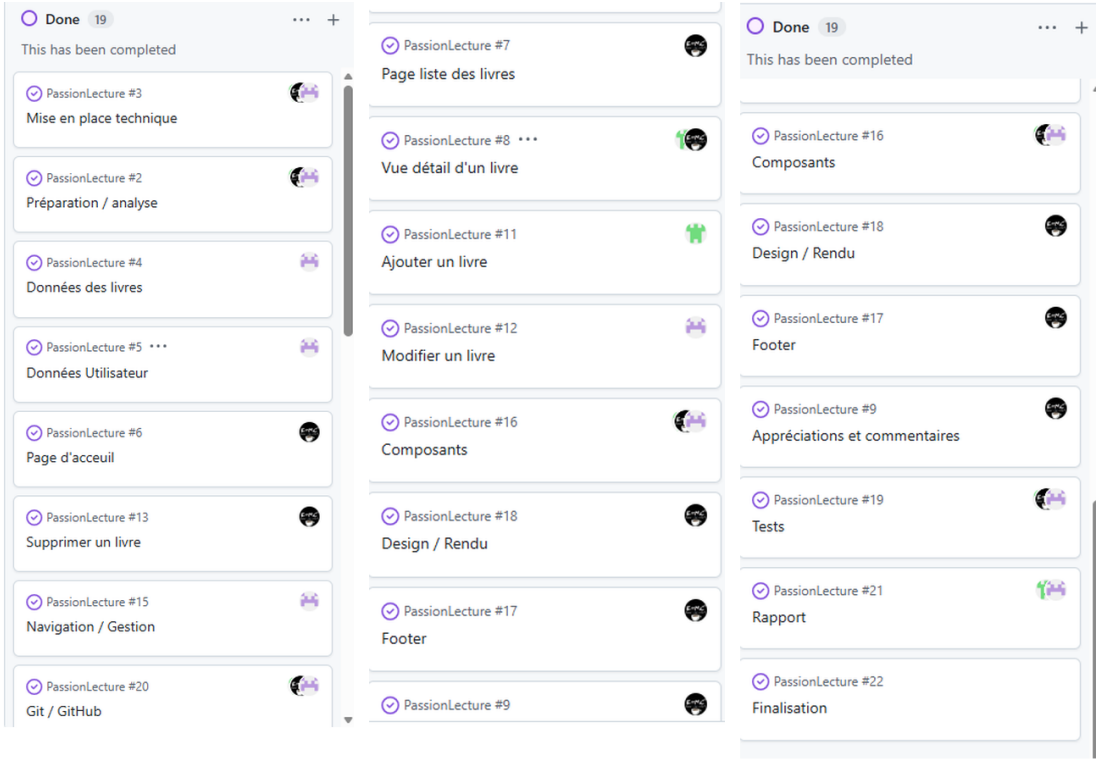


Figure 3 – Extraits du suivi GitHub Projects avec les affectations.

Personne	Responsabilités visibles dans GitHub Projects
Arnaud Thill	Données des livres et des utilisateurs ; modification ; navigation. Partage de l'analyse, des composants, de Git/GitHub et du rapport.
Kentin Fankhauser	Accueil ; catalogue ; suppression ; appréciations/commentaires ; footer ; design. Partage de la fiche détail, des composants et de Git/GitHub.
Sarangan Shanthakumar	Ajout d'un livre. Partage de la fiche détail, des composants, de l'analyse, de Git/GitHub et du rapport.
Travail collectif	Mise en place technique et tâches communes ; finalisation #22 sans attribution visible. La carte #19 est classée « Done » dans le tableau.

2.3 Organisation du code

Nous avons utilisé Vue.js 3 et Vite pour notre projet. Nous avons organisé le code en séparant les pages, les composants réutilisables, les routes et les échanges avec l'API simulée. Cette structure nous a facilité la répartition des tâches et le travail en équipe.

```
Projet294/
  src/
    main.js                 Démarrage de Vue
    App.vue                 Structure générale
    router/index.js         Définition des routes
    views/                  Pages de l'application
    components/             Composants réutilisables
    services/               Requêtes vers l'API
  public/                   Images et extraits PDF
  db.json                   Données de démonstration
  package.json              Dépendances et scripts
```

Élément	Fonction
Vues Vue	Accueil, catalogue, détail, ajout, modification, liste personnelle et formulaires de compte.
Composants	En-tête, pied de page et affichages réutilisables des ouvrages.
Vue Router	Navigation entre /, /books, /books/:id, /myBooks, /login et les formulaires.
Services Axios	Appels HTTP sur les collections du serveur JSON local.

Nous avons défini nos données dans db.json, avec notamment les collections de livres, d'utilisateurs, de genres et d'appréciations. Nous utilisons des services qui effectuent des requêtes GET, POST, PUT et DELETE vers une API locale. Notre frontend ne dépend donc pas encore d'un backend métier définitif.

3. Réalisation

3.1 Technologies et navigation

Nous utilisons Vue.js, Vue Router, Git/GitHub et JSON Server, conformément au cahier des charges. Vite sert à lancer le projet et Axios à envoyer les requêtes HTTP. Nous avons regroupé l'en-tête, RouterView et le pied de page dans App.vue, puis associé les URL aux vues dans router/index.js.

3.2 Page d'accueil

Dans HomeView.vue, nous chargeons les livres, les genres et les appréciations. Une propriété calculée nous permet d'afficher cinq ouvrages à l'accueil, selon l'ordre reçu, sans tri explicite par date d'ajout.

3.3 Catalogue par catégorie

Dans BooksCatalogueView.vue, nous chargeons les livres et les genres, puis affichons les ouvrages avec BookCataloguePreview.vue.

3.4 Vue détaillée et appréciations

Dans BookDetailView.vue, nous récupérons l'identifiant du livre depuis l'URL et appelons le service correspondant. Nous avons prévu la couverture, le résumé, les informations du livre, un lien vers un extrait, les avis et une note de 0 à 5. Certains contenus restent écrits directement dans la vue et nous n'avons pas encore ajouté la sauvegarde persistante des avis.

3.5 Ajouter un livre

Nous avons créé un formulaire dans AddBooksView.vue : il envoie une requête POST via BookService.js et charge les genres depuis le service associé. Nous avons constaté une différence entre les champs numberOfPage et numberOfPages.

3.6 Modifier un livre

Dans UpdateBookView.vue, nous chargeons le livre, préremplissons le formulaire, puis envoyons les modifications par PUT avant de revenir à « Mes livres ».

3.7 Supprimer un livre

Dans MyBooksView.vue, nous affichons les livres et utilisons MyBookPreview.vue pour les modifier ou les supprimer. Nous envoyons une requête DELETE puis actualisons la liste. Nous devons encore limiter celle-ci aux livres du membre connecté.

3.8 Gestion des comptes, des droits et pied de page

Nous avons relié le formulaire d'inscription à l'URL /users pour envoyer les informations saisies. Nous avons aussi prévu un écran de connexion, mais nous traiterons la gestion de session, les restrictions d'accès et les privilèges dans le module 295. Nous devons également faire apparaître les noms des trois créateurs dans le pied de page, comme le demande le cahier des charges.

3.9 Organisation du groupe avec Git/GitHub

Nous avons réparti nos tâches dans GitHub Projects et nous avons travaillé sur un dépôt Git partagé. Nous nous sommes occupés de différentes parties de l'application tout en collaborant sur des éléments communs, notamment la structure technique, les composants et la navigation. Grâce à Git, nous avons pu conserver les versions du code et regrouper nos contributions.

Difficultés rencontrées – proposition à confirmer : en travaillant simultanément, nous avons dû tenir compte des risques de conflits de fusion sur les fichiers communs. Nous avons notamment relevé des marqueurs de conflit Git dans le fichier db.json de notre archive initiale. Nous avons aussi constaté des différences de noms de propriétés entre les formulaires et le JSON, qui peuvent compliquer l'intégration. Pour limiter ces problèmes, nous aurions dû définir plus tôt des conventions communes et échanger régulièrement avant les fusions.

4. Conclusion

4.1 Bilan général

Avec Passion Lecture, nous avons mis en pratique les connaissances du module 294 en réalisant le frontend d'une application web consacrée aux livres. Nous avons conçu les maquettes, organisé notre code en vues et composants, mis en place la navigation avec Vue Router et utilisé une API REST simulée pour gérer les données. Nous nous sommes appuyés sur Git et GitHub Projects pour répartir les responsabilités et suivre notre avancement.

Sur nos dernières captures, nous comptons 19 tâches classées comme terminées. Certaine cependant reste a terminer, notamment la gestion des permissions et certains formulaires. Nous avons ainsi posé une base frontend que nous pourrons relier au backend du module 295.

4.2 Conclusions personnelles

4.2.1 Arnaud Thill

Personnellement, j'ai trouvé que ce projet s'est très bien déroulé dans l'ensemble. La répartition du travail au sein du groupe s'est bien passée et a été équitable. Chacun a pu participer aux différentes tâches et apporter sa contribution, ce qui nous a permis d'avancer efficacement et de travailler dans de bonnes conditions.

J'ai également beaucoup apprécié la manière dont nous avons utilisé Git et GitHub pour gérer notre code. Le fait de travailler avec différentes branches, puis de fusionner notre travail avec des merge, était très intéressant. Cela m'a permis d'apprendre beaucoup de choses sur Git et de mieux comprendre son fonctionnement. J'ai notamment pu constater à quel point cet outil est pratique et utile lorsqu'on travaille à plusieurs sur un même projet.

Concernant le projet en lui-même, j'ai trouvé intéressant de développer une application avec Vue.js. C'était une bonne occasion de mettre en pratique les connaissances acquises et de découvrir davantage les possibilités offertes par ce framework. J'ai apprécié le fait de pouvoir construire les différentes pages et fonctionnalités d'une application complète tout en organisant correctement le code.

Pour conclure, je retiens une expérience très positive de ce projet. Il m'a permis de progresser techniquement, notamment sur Vue.js et Git, mais aussi de développer ma manière de travailler en équipe. Ce sont des compétences que je pourrai facilement réutiliser dans de futurs projets.

4.2.2 Sarangan Shanthakumar

Ce projet m'a permis de mieux comprendre le développement d'une application Vue.js en équipe. J'ai notamment participé à la fonctionnalité d'ajout de livre, à la vue détaillée et au rapport. J'ai constaté qu'une fonctionnalité ne dépend pas uniquement du formulaire visible : les données, les services et la navigation doivent être cohérents pour qu'elle fonctionne correctement.

Avec du recul, j'aurais pu consacrer davantage de temps à la coordination des modifications et à l'intégration des différentes parties. Je retiens surtout l'importance de communiquer clairement, de suivre les conventions du projet et de ne pas

considérer qu'une tâche est aboutie uniquement parce que son interface est créée. Ces points me serviront dans de futurs projets réalisés en groupe.

4.2.3 Kentin Fankhauser

Ce projet m'a fait découvrir Vue.js et ses spécificités. Certaines m'ont demandé un temps d'adaptation, mais j'ai trouvé cette découverte intéressante. Grâce à Passion Lecture, j'ai mieux compris comment les pages, les composants et la navigation s'assemblent pour former un frontend.

Au final, j'ai trouvé ce projet bénéfique, car il m'a donné une vue d'ensemble plus concrète du frontend d'un site de type blog de lecture. J'ai mieux compris les étapes de création d'une interface web et ce que Vue.js peut apporter à ce travail. Malgré les particularités du framework, je retiens une expérience enrichissante qui me sera utile pour de futurs projets.

4.3 Critique constructive de la planification

Nous avons utilisé GitHub Projects pour suivre nos tâches et répartir le travail entre nous trois. Nous avons regroupé dans un même tableau les étapes de préparation, de création des données, de développement des vues et de finalisation. Cette organisation nous a permis de travailler sur plusieurs fonctionnalités en parallèle.

Avec du recul, nous aurions dû définir plus précisément les priorités et les dépendances entre nos tâches, car certaines vues nécessitaient des données et des services déjà cohérents. Nous aurions également gagné à fixer des échéances intermédiaires et à échanger plus régulièrement avant nos fusions Git. Lors d'un prochain projet, nous définirions des priorités plus claires avant de développer les différentes vues.

5. Webographie, bibliographie et glossaire

5.1 Sources du projet

- ETML, P_WEB294 – Cahier des charges « Passion lecture », version 1.2 du 04.02.2026.
- Maquettes du projet : WireFrame.png.
- Tableau GitHub Projects : captures du 30 octobre 2026.
- Code source du projet : archive Projet294.zip (fichiers Vue, services et db.json).

5.2 Webographie externe

À compléter : références des documentations que nous avons effectivement consultées.

5.3 Glossaire

Terme	Définition
Frontend	Partie visible et interactive de l'application web.
Vue.js	Framework JavaScript utilisé pour construire les interfaces.
Vue Router	Bibliothèque de navigation entre les pages Vue.
API REST	Interface permettant d'échanger des

Terme	Définition
	données via HTTP.
JSON Server	Serveur simulant une API à partir de données JSON.
Git/GitHub	Outils de gestion des versions et de collaboration.